OPERATION KIVOTOS · SECTOR 작업 사전점검 · 구조 도식

몬스터 · 스포너 — 지금 있는 구조 그대로

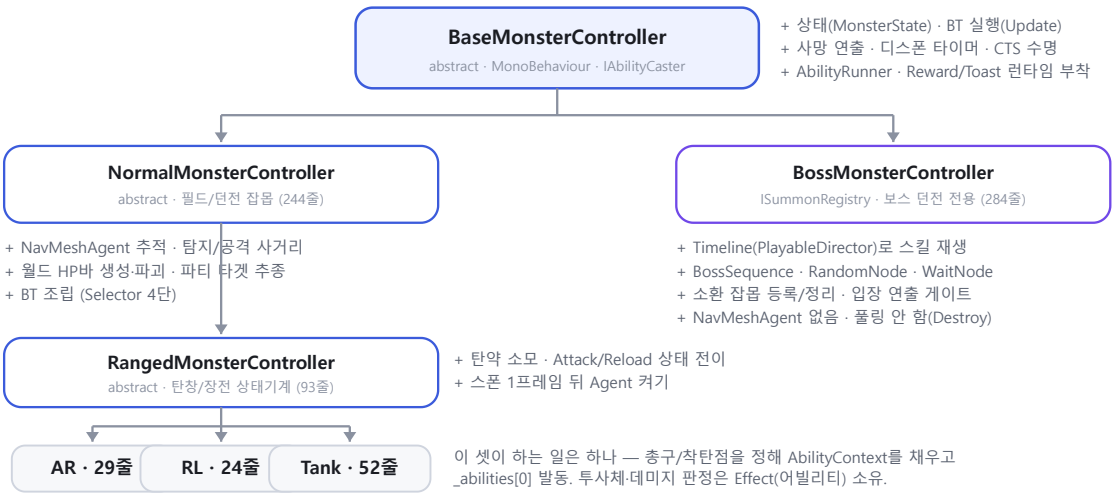
Sector를 손대기 전에, 그 아래에서 실제로 돌아가고 있는 것을 그림으로 고정한다. **결함·개선안은 이 문서에 없다** — 별도 문서(Issues.md)로 분리했다.

읽는 법
파란색 = 매 프레임 도는 것(Update·BT) · 보라색 = 비동기(UniTask·Addressables) · 초록색 = 데이터/에셋 · 주황색 = 몬스터 바깥의 구독자. 실선 화살표는 **직접 호출**, 점선 화살표는 **이벤트 구독**이다.

1. 타입 계보 — 층마다 무엇을 더하는가

컨트롤러는 4계층이고, 보스만 2계층에서 갈라져 완전히 다른 길을 간다.

① 상속 구조와 각 층의 추가분

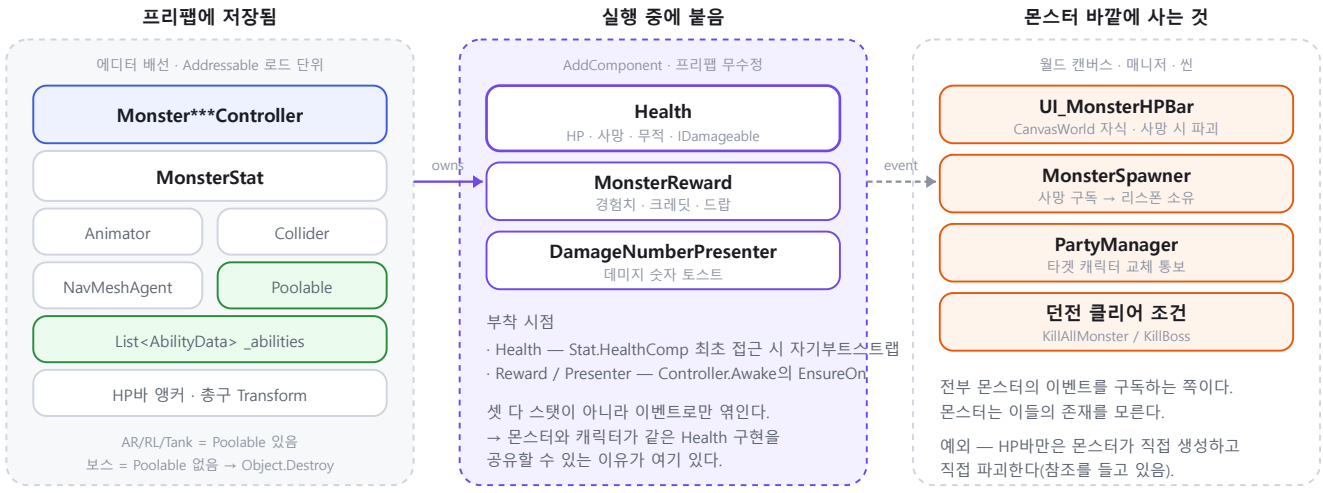


잎 클래스가 얇다 = 전투의 실체는 컨트롤러가 아니라 AbilitySystem에 있다. 보스만 이 원칙에서 벗어나 Timeline을 직접 조작한다.

2. 몬스터 한 마리의 구성 — 누가 무엇을 소유하는가

프리팹에 박혀 있는 것 / 실행 중에 붙는 것 / 바깥에서 구독하는 것, 세 덩어리로 나뉜다.

② 컴포넌트 소유도 GameObject 1기 기준

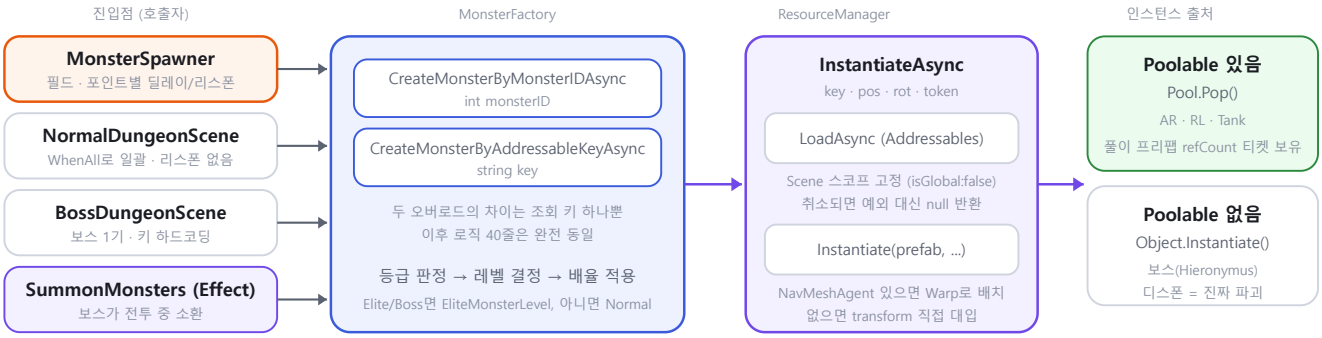


HP·사망·무적은 Stat이 아니라 Health가 소유한다(StatRefactor 3단계). 보상·데미지 표시는 Health 이벤트의 구독자로 밀려나 있다.

3. 스폰 파이프라인 — 진입점 4개가 한 팩토리로 모인다

필드(스포너)·일반던전·보스던전·보스소환. 넷 다 MonsterFactory를 통과하지만, **스폰 정책(딜레이·리스폰)은 스포너에만 있다.**

③ 생성 경로 호출 → 팩토리 → 리소스



인스턴스 1기가 완성되기까지 — 실제 실행 순서



③과 ④의 순서에 주의 — 몬스터는 **스탯이 주입되기 전에 이미 활성화되어 있다.** OnEnable은 직전 생애의 MaxHp로 HP를 채우고, 그 다음 SetStat이 값을 덮어쓴 뒤 다시 회복시킨다. 같은 프리팹 = 같은 종이라 현재는 값이 일치한다.

④의 SetStat은 `OnMonsterDead = null` 로 이전 생애의 구독을 지우고 시작한다 → 그래서 ⑤에서 스포너가 다시 구독한다.

던전 두 곳은 팩토리를 직접 호출하고 끝난다. 딜레이·리스폰·취소 토큰은 스포너만 가진 개념이다.

4. 스포너 내부 — 포인트 인덱스가 유일한 키

스포너는 자식 Transform을 그대로 스폰 포인트로 삼고, 모든 상태를 그 인덱스로 정렬한다.

④ 슬롯 자료구조와 슬롯 상태 전이

Awake — 자식 Transform이 곧 스폰 포인트다

MonsterSpawner (GameObject) — 자식 N개 = 스폰 포인트 N개

index	_spawnPoints	_respawnCts	_spawnedMonsters
0	Transform	null	살아있는 몬스터
1	Transform	CTS (대기중)	null
2	Transform	null	null (시체 연출 중)
...			

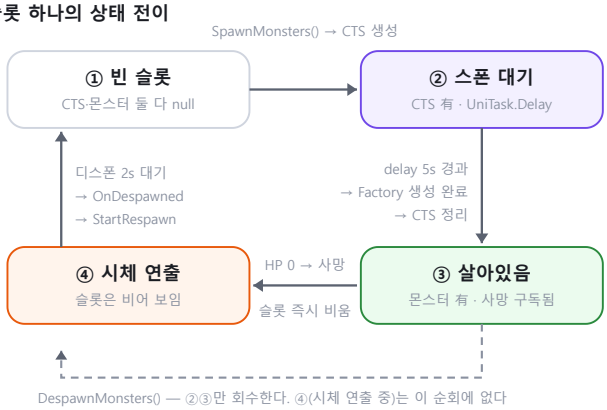
두 칸이 모두 null = "비었다"와 "시체 연출 중"이 구분되지 않는다

같은 사실을 담는 그릇이 5개

- _spawnPoints : List<Transform> — 위치
- _respawnCts[] · _spawnedMonsters[] — 인덱스 기준
- _activeMonsters : List — 디스폰 순회용
- _monsterToPointIndex · _monsterToSpawnInfo : Dict — 몬스터 기준

사망 처리는 이 다섯 곳을 손으로 동기화한다 (Handle 한 함수에서 4번 갱신)

슬롯 하나의 상태 전이



한 바퀴에 걸리는 시간

사망 → (디스폰 2s) → (리스폰 delay 5s) → 재등장 = 약 7초

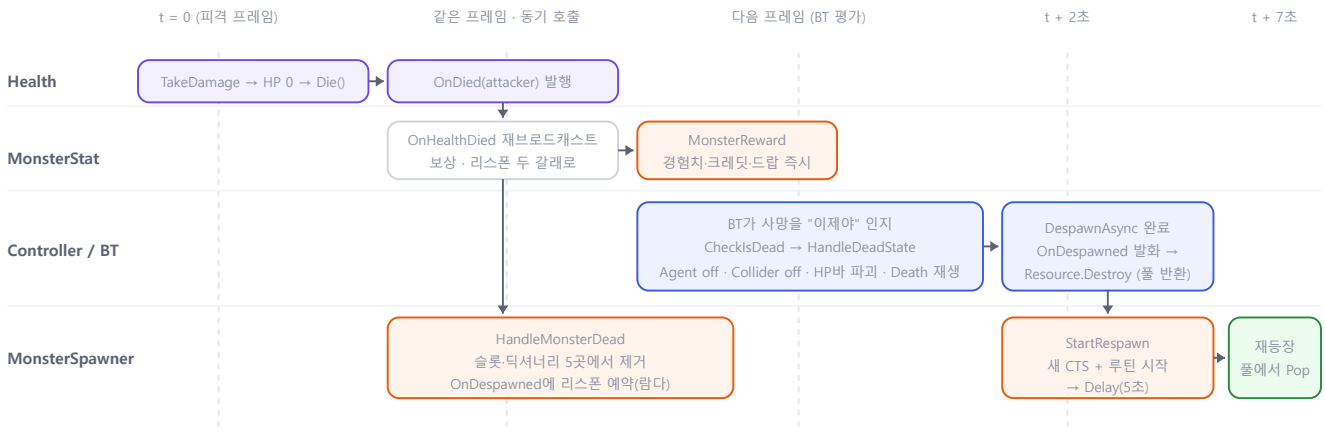
delay 값은 SpawnerData.json에 있고, 최초 스폰 지연과 리스폰 간격을 같은 필드 하나가 겸한다 (현재 전 항목 5.0)

스포너는 "몬스터가 없어도 리스폰 타이머만 도는" 대표값 시뮬레이션 구조다. 몬스터 없이도 슬롯은 살아있다.

5. 사망에서 재등장까지 — 시간 축

사망 "통보"는 즉시, 사망 "연출"은 다음 프레임에 시작한다. 이 둘이 다른 시점이라는 게 이 구조의 핵심이다.

⑤ 죽음 → 리스폰 시퀀스 약 7초



· 보상은 **피격 프레임에 즉시** 지급된다. 사망 연출과 무관하다.
· 스포너는 사망 시점에 슬롯을 비우지만, 실제 몬스터는 2초 더 씬에 남아 있다 — 이 2초 동안 슬롯은 "빈 슬롯"과 구분되지 않는다.

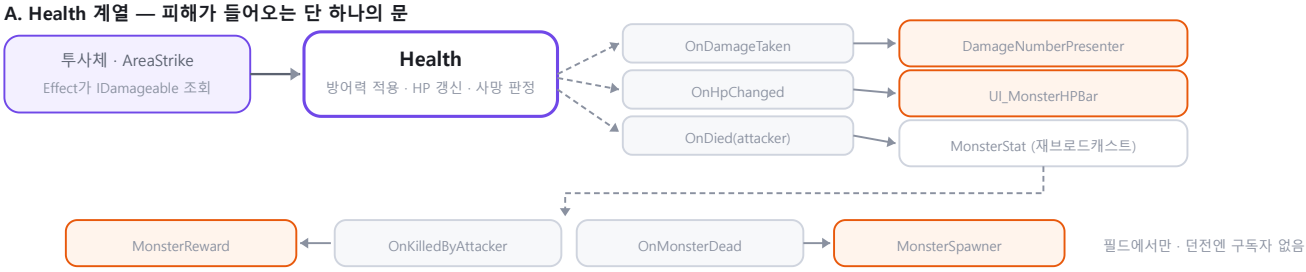
HP 0 → 재등장까지 세 개의 서로 다른 시계가 붙어 있다: 즉시(이벤트) · 다음 프레임(BT) · 초 단위(UniTask.Delay).

6. 이벤트 배선 — Health가 허브다

몬스터에서 밖으로 나가는 신호는 두 계열이다. Health 계열(피해·죽음)과 Controller 계열(연출 끝·클리어).

⑥ 누가 무엇을 구독하는가

A. Health 계열 — 피해가 들어오는 단 하나의 문



B. Controller 계열 — 연출이 끝났음을 알리는 신호



사망 신호가 두 계열로 갈라져 있는 이유 — 보상은 **죽는 순간**에, 리스폰은 **시체가 사라진 뒤**에 일어나야 하기 때문이다.

몬스터 자신은 보상·UI·리스폰을 전혀 모른다. 전부 이벤트 구독자 쪽에 있다.

7. 행동 트리 — 일반 몬스터와 보스는 다른 트리를 쓴다

노드 구현은 공용(BTNode.cs 214줄)이고, 조립만 컨트롤러가 다르게 한다. 평가는 매 프레임 `Update`에서 루트부터.

⑦ 트리 구조 비교

일반 몬스터 (NormalMonsterController)

- Selector — 위에서부터 하나 성공할 때까지
  - ① Sequence — 사망
    CheckIsDead → HandleDeadState
  - ② Sequence — 전투 (RangedMonsterController가 제공)
    CheckAttackRange → HandleCombat (탄약 있으면 공격, 없으면 장전)
  - ③ Sequence — 추적
    CheckDetectRange → TrackTarget (0.5m 이상 움직였을 때만 SetDestination)
  - ④ ActionNode — Idle

노드는 전부 무상태(Selector·Sequence·ActionNode) → 풀에서 재사용해도 이전 생애가 남지 않는다. 상태를 들고 있는 노드(WaitNode·BossSequence·RandomNode)는 보스 트리에만 쓰인다.

보스 (BossMonsterController)

- Selector
  - ① Sequence — 사망 (일반과 동일 + 소환수 정리)
    CheckIsDead → HandleDeadState (Timeline 강제 정지 · 소환수 즉사)
  - ② BossSequence — 진행 단계를 기억하는 시퀀스
    - CheckEntranceFinished — 입장 연출이 끝났는가
    - RandomNode — 스킬 0~4 중 하나 추첨 → Timeline 재생
    - WaitNode(_patternInterval = 2초)

보스는 추적·사거리 개념이 없다. 스킬 재생이 끝났는지는 PlayableDirector.stopped 이벤트가 알려주고, 그 신호가 _isActionRunning 플래그를 내려 BT가 다음 단계로 넘어간다 (BT ↔ Timeline 이중 상태).

일반 몬스터는 "조건 → 행동"의 평평한 우선순위 트리, 보스는 "연출 재생 → 대기"의 진행형 트리다.

8. Sector와 스포너의 관계

스포너는 씬 계층 구조상 Sector의 자식이고, Sector는 트리거 볼륨 하나다.

⑧ 계층 구조와 활성 전환

씬 계층 (BackStreetMap.prefab)

- BackStreetMap — Sector 5개 · MonsterSpawner 23개
  - Sector (BoxCollider isTrigger · _sectorID)
    - MonsterSpawner (_spawnerId · _mapId)
      - 스폰 포인트 (빈 Transform) × N

Sector는 Start에서 자식 스포너를 수집하고 매니저에 등록한다.
스포너 목록은 이때 한 번만 만들어진다 (_isInitialized).

활성 전환 (한 번에 한 섹터)

플레이어 트리거 진입 → SectorManager.OnPlayerEnterSector

OnTriggerExit — 비어 있음
이탈은 아무 일도 하지 않는다

→ 이전 활성 섹터 → DeactivateSector()
스포너 전부 DespawnMonsters()

BenchmarkTester F1 / F2
전 섹터 일괄 켜기 / 끄기

→ 새 섹터 → ActivateSector()
스포너 전부 SpawnMonsters()

DeactivateSector는 GameObject를 끄지 않는다 — 스포너에게 "몬스터를 회수하라"고 시킬 뿐이다.
즉 비활성 섹터의 스포너도 계속 살아있는 컴포넌트다.

활성 판정의 단위는 섹터(볼륨)이고, 실행의 단위는 스포너다. 이 두 단위가 지금 1:다 로 묶여 있다.

9. 실물 인벤토리

구분	파일	줄수	역할 한 줄
컨트롤러	BaseMonsterController.cs	166	상태·BT 실행·사망 연출·디스폰·CTS
	NormalMonsterController.cs	244	추적·HP바·BT 조립
	RangedMonsterController.cs	93	탄창·장전 상태기계
	AR / RL / Tank	29 / 24 / 52	총구·착탄점 정해 어빌리티 발동
	BossMonsterController.cs	284	Timeline 스킬·소환·입장 연출
행동	BTNode.cs	214	Selector·Sequence·Random·Wait·BossSequence
스탯	BaseStat / MonsterStat / Health	94 / 91 / 84	속성 보관 / 몬스터 고유값·재브로드캐스트 / HP·사망·피격
부가	MonsterReward / DamageNumberPresenter	56 / 51	보상 지급 / 데미지 숫자
스폰	MonsterFactory.cs	97	데이터 조회 → 생성 → 스탯 주입
	MonsterSpawner.cs	209	포인트별 스폰·리스폰·취소
	Sector.cs / SectorManager.cs	81 / 70	트리거 볼륨 / 활성 섹터 1개 추적
데이터	SpawnerData.json / MonsterDatabaseSO	—	스포너별 스폰 목록 / 몬스터·레벨·맵 테이블

이 문서에 없는 것

결함·리스크·개선안은 전부 **Issues.md**에 있다. 이 문서는 "지금 이렇게 되어 있다"만 기술하며, 잘잘못을 판단하지 않는다.